Zonoth

SESSION 04 · 2H

Docker Compose

Every flag you typed by hand in Lab 03 — declared once, in one file, under version control.

The Compose file | The Compose CLI | Developer loop | Bridge to Kubernetes | Lab 04

SESSION 04

What we'll cover today

1 · The Compose file

The Compose Specification: services, networks, volumes, `build` vs `image`, environment and interpolation, `depends_on` with health conditions.

2 · The Compose CLI

Ports and the project network, `up -d --wait`, `ps`, `logs -f`, `exec`, `--scale`, profiles, `down` vs `down -v`.

3 · Loop & limits

`docker compose watch`, override files, `compose config`, Compose in CI — and where Compose runs out of road.

4 · Lab 04

Pinboard in one file: healthchecks, `depends_on` conditions, `.env`, live editing, three API replicas.

01

The Compose file

One declarative YAML file describing a whole application — services, networks and volumes together.

MOTIVATION

Why Compose

Lab 03, by hand

- Three long `docker run` lines, in the right order
- Network and volume created separately, first
- Nothing waits for Postgres to be ready
- Teardown is a checklist you will get wrong
- The knowledge lives in your shell history

With Compose

One file describes every container, network and volume — with dependencies and health conditions.

BASH

```
docker compose up -d --wait
docker compose down -v
```

Reviewable in a pull request. Identical on every laptop and in CI.

THE FORMAT

The Compose Specification

- An open spec, versioned by the spec itself — **no** `version:` **key**. If you see one, the file is from 2020.
- Default filenames: `compose.yaml` (preferred), then `docker-compose.yml`
- `name:` sets the project name — the prefix on every container, network and volume
- Top-level keys: `services`, `networks`, `volumes`, `configs`, `secrets`

✓ Modern

- `docker compose up` (a space)
- `compose.yaml`, no `version:`
- `name: pinboard`

▲ Legacy — don't copy from old blogs

- `docker-compose up` (a hyphen)
- `version: "3.8"`
- `links:`, `COMPOSE_FILE` gymnastics

ANATOMY

Services, volumes, networks

YAML

```yaml
name: pinboard
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-pinboard}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-pinboard-secret}
      POSTGRES_DB: ${POSTGRES_DB:-pinboard}
    volumes:
      - pinboard-data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro

volumes:
  pinboard-data:
```

WHERE THE IMAGE COMES FROM

`build` vs `image`

- `image:` alone — pull and run
- `build:` alone — build an anonymous image
- **Both** — build, and tag the result with that name
- `docker compose build`, or `up --build` to force a rebuild

YAML

```yaml
services:
  api:
    build:
      context: ./pinboard-api
      args:
        APP_VERSION: "1.0"
    image: pinboard-api:1.0

  web:
    build: ./pinboard-web
    image: pinboard-web:1.0
```

Always set `image:` next to `build:` — you get a predictable tag you can push, scan, or `kind load` later.

CONFIGURATION

environment, env_file, interpolation

YAML

```yaml
services:
  api:
    env_file:
      - .env                        # read by Compose AND passed to the container
    environment:
      APP_THEME: ${APP_THEME:-emerald}
      APP_GREETING: ${APP_GREETING:-Welcome to Pinboard (Compose)}
      DATABASE_URL: postgres://${POSTGRES_USER}:${POSTGRES_PASSWORD}@db:5432/${POSTGRES_DB}
```

- `${VAR}` is substituted by **Compose**, from the shell and from `.env`, before the file is parsed
- `${VAR:-default}` falls back; `${VAR:?message}` fails loudly if it is unset
- To pass a literal `$` into the container, double it: `$${POSTGRES_USER}`
- `.env` holds credentials, is git-ignored, and ships as `.env.example`

STARTUP ORDER

depends_on + healthcheck

YAML

```yaml
services:
  db:
    image: postgres:17-alpine
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 3s
      retries: 10

  api:
    image: pinboard-api:1.0
    depends_on:
      db:
        condition: service_healthy
```

⚠ Plain `depends_on: [db]` only waits for the container to **start**. "Started" is not "accepting connections".

LIVE DEMO

Pinboard's compose.yaml, from scratch

Grow the stack one service at a time — db, then api, then web — with `docker compose up -d --wait`, and open the app — no `docker run` in sight.

What you'll see · the file grow service by service, `--wait` blocking on the DB health check

Files · demos/session04/compose.step1.yaml, compose.step2.yaml, compose.yaml

Duration · 8 min

02

Networking, ports and the CLI

How services reach each other, and the ten commands you will use every day.

DISCOVERY

`ports` vs `expose`

- Compose creates `pinboard_default`, a user-defined bridge — so DNS by **service name** works out of the box
- `ports` publishes to the host: the only door from your browser
- `expose` is documentation between services
- Add a `networks:` block only when you want isolation, e.g. a backend network the web tier cannot reach

YAML

```yaml
services:
  api:
    expose:
      - "8080"          # api:8080, internal only

  web:
    environment:
      API_URL: http://api:8080
    ports:
      - "8080:8080"     # host:container
```

RUNTIME POLICY

restart and resource limits

- `restart:` is the same policy set as `docker run --restart`
- Limits live under `deploy.resources` — honoured by `docker compose` on a single host
- Other `deploy` keys (`replicas`, placement, update policy) are Swarm territory
- Still no self-healing: a limit that is hit kills the container, and only the restart policy brings it back

YAML

```yaml
services:
  api:
    restart: unless-stopped
    deploy:
      resources:
        limits:
          memory: 128M
          cpus: "0.50"
```

DAILY DRIVER

The commands you'll actually type

BASH

```
docker compose up -d --wait          # create everything, block until healthy/started
docker compose ps                    # state, health, published ports
docker compose logs -f api           # follow one service (no -f: dump and exit)
docker compose exec db psql -U pinboard -d pinboard
docker compose restart api
docker compose stop                  # keep containers, keep data
docker compose down                  # remove containers + the project network
docker compose down -v               # …and named volumes: the notes are gone
```

⚠ `down -v` is the destructive one. `down` keeps `pinboard-data`; `down -v` deletes it.

MORE OF THE SAME THING

Scaling a service

BASH

```
docker compose up -d --scale api=3 --wait
docker compose ps
# pinboard-api-1  pinboard-api-2  pinboard-api-3

for i in $(seq 6); do
  docker compose exec -T web wget -qO- http://api:8080/api/info
done   # the "hostname" field rotates between the three containers
```

- Compose DNS returns all three container IPs for `api` — clients round-robin
- A service with a fixed `ports` mapping cannot be scaled: only one container can own host port 8080
- This is manual scaling on one machine. Nothing rebalances, nothing reacts to load

OPTIONAL PIECES

Profiles

- Services in a profile are **not started** unless the profile is on
- Keep debug tools, seeders and test runners in the same file without slowing everyone down
- Enable with `--profile` or the `COMPOSE_PROFILES` environment variable

YAML

```yaml
services:
  adminer:
    image: adminer:5
    ports:
      - "8081:8080"
    depends_on:
      - db
    profiles:
      - tools
```

BASH

```bash
docker compose --profile tools up -d
COMPOSE_PROFILES=tools docker compose up -d
```

LIVE DEMO

Scale the API to three

One flag, three containers — and nginx round-robins between them. Watch the hostname in `/api/info` change on every refresh.

What you'll see · `--scale api=3`, rotating hostnames, one container stopped and the load moving — then `down` vs `down -v`

Files · demos/session04/compose.yaml

Duration · 10 min

03

The developer loop — and the ceiling

Live reload, override files, CI — then an honest list of what Compose will never do for you.

INNER LOOP

`docker compose watch`

- `sync` — copy changed files into the running container
- `rebuild` — rebuild the image and replace the container
- `sync+restart` — copy, then restart the process (config files)
- Run it with `docker compose watch`; it takes over the terminal and streams what it does

YAML

```yaml
services:
  web:
    develop:
      watch:
        - action: sync
          path: ./pinboard-web/html
          target: /usr/share/nginx/html
        - action: rebuild
          path: ./pinboard-web/Dockerfile
  api:
    develop:
      watch:
        - action: rebuild
          path: ./pinboard-api
```

LIVE DEMO

Edit the page, see it instantly

`docker compose watch` syncing the Pinboard web folder — then touch the Dockerfile and watch it rebuild.

What you'll see · sync in milliseconds, rebuild in seconds

Files · demos/session04/compose.override.yaml, src/pinboard-web/html

Duration · 6 min

COMPOSITION

Multiple files & overrides

- `compose.yaml` + `compose.override.yaml` are merged **automatically**
- `-f` takes explicit control — first file is the base, later files override
- Scalars are replaced, mappings are merged, most sequences are appended
- Paths stay relative to the **first** file

BASH

```
# dev: base + override, automatically
docker compose up -d --wait

# prod-ish: base + an explicit second file
docker compose \
  -f compose.yaml \
  -f compose.prod.yaml \
  up -d --wait
```

Keep the base file environment-neutral. Put ports, bind mounts and `develop:` rules in the override.

TRUST, THEN VERIFY

`compose config` renders the truth

BASH

```
docker compose config                  # merged + interpolated + validated
docker compose config --services       # just the names
docker compose config --images         # what would be pulled or built

docker compose -f compose.yaml -f compose.prod.yaml config > rendered.yaml
```

- Shows the file **after** variable substitution — the fastest way to catch an empty `${POSTGRES_PASSWORD}`
- Fails with a clear error on invalid YAML or unknown keys: a free lint step in CI
- Reveals exactly how your override files merged, so you stop guessing

LIVE DEMO

Config, overrides and profiles

Render the file Compose actually assembled, switch the override off with `-f`, and run a profiled one-shot `psql` container on demand.

What you'll see · interpolated values and merged `develop:` rules, `config --services`, `run --rm dbtools`

Files · demos/session04/compose.yaml, compose.override.yaml

Duration · 3 min core · 6 with profiles

AUTOMATION

Compose in CI

BASH

```
docker compose config -q                    # lint the file
docker compose build --pull
docker compose up -d --wait                 # non-zero exit if never healthy
docker compose run --rm --no-deps tests     # exit code = test result
docker compose logs --no-color > compose-logs.txt
docker compose down -v --remove-orphans     # always, even on failure
```

- `--wait` plus health checks removes every `sleep 30` from your pipeline
- Alternative: `up --abort-on-container-exit --exit-code-from tests`
- Set `name:` or `-p` per build so parallel jobs never collide

THE CEILING

What Compose cannot do → Kubernetes

WHAT YOU NEED	COMPOSE	KUBERNETES
Run across many machines	One Docker host. Full stop.	A scheduler places Pods on any node in the cluster
Survive a machine dying	Restart policy — on that same host	Controllers reschedule the workload elsewhere
Zero-downtime updates	`up` stops the old container, then starts the new one	Deployment rolling update, readiness gates, `rollout undo`
Scale with demand	`--scale api=3`, typed by a human	HorizontalPodAutoscaler on CPU or custom metrics
Secrets at scale	A `.env` file on someone's laptop	Secret objects, RBAC, encryption at rest, external stores



Compose is excellent for **one machine**: development, CI, small single-host deployments. Everything above needs an orchestrator.

THE LANDSCAPE

Where Swarm, Nomad and Kubernetes fit

Docker Swarm

Built into the engine, reads Compose files with `docker stack deploy`. Easiest step up — but a small ecosystem and little momentum.

HashiCorp Nomad

One binary, schedules containers **and** plain binaries or VMs. Simple and solid; a niche compared to K8s.

Kubernetes

The de-facto standard: every cloud offers it managed, and the whole CNCF ecosystem targets its API. Sessions 05–10.

Compose files do not become Kubernetes manifests. Converters like `kompose` give you a starting point, never a production result.

LAB 04 · 30–40 MIN

Pinboard in one file

Turn yesterday's wall of `docker run` flags into a `compose.yaml` with health checks and dependency conditions, bring it up with one command, and edit the site live.

You'll practise: `compose.yaml` · `.env` · `healthcheck` · `depends_on: service_healthy` · `up -d --wait` · `compose watch` · `--scale api=3` · `down` vs `down -v`

Key takeaways

✓ One file, one command

The Compose Specification — no `version:`, `docker compose` with a space, `up -d --wait`.

✓ Health drives startup

`healthcheck` plus `depends_on: condition: service_healthy` ends the startup race.

✓ Great on one host

`watch`, overrides and `config` for the dev loop and CI. Multi-host, self-healing and autoscaling are out of scope.

✓ Next up

Session 05 — Kubernetes architecture, `kubectl` and Pods.

Zonoth

Questions?

Up next — Session 05: Kubernetes — architecture, kubectl, Pods. One host was never going to be enough.